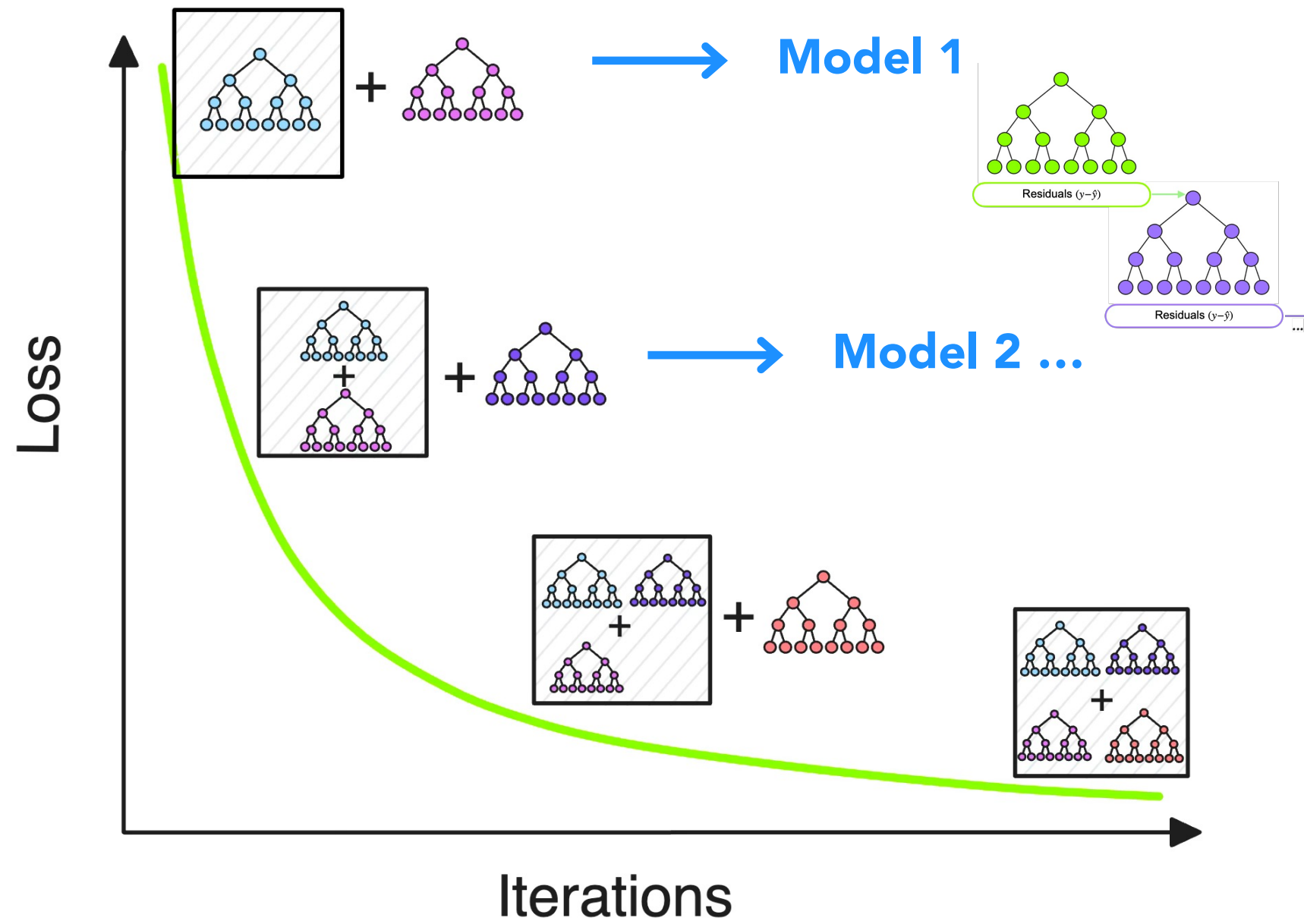


Boosted Tree Interactions to WOE

A New Approach to Feature Engineering in Credit Scoring



Gradient Boosted Decision Trees (GBDT) are a family of machine learning algorithms used for regression, classification, and scoring (rank ordering or probability estimation) tasks

Each tree **cumulatively** contributes evidence to the final prediction by modeling residuals of the previous trees

Trees with higher depths include **interactions between features**, which is useful for feature engineering

Each tree with interactions can be viewed as a scoring model that can be **linearized** by treating tree leaf indices as bins to calculate WOE features that feed into a logistic regression scorecard

This deck demonstrates the feature engineering approach in practice →

Boosted Tree Interactions to WOE

A New Approach to Feature Engineering in Credit Scoring



Each tree in the ensemble (e.g., XGBoost, LightGBM, CatBoost) gets a specific index where the observations lands, *here we use a depth of 3*

	tree_0	tree_1	tree_2	tree_3	tree_4	tree_5	tree_6	tree_7	tree_8	tree_9	tree_10
obs ₁	12	6	12	12	12	14	14	12	11	14	14
obs ₂	8	8	7	8	7	7	9	7	8	7	8
obs ₃	5	6	5	5	5	12	13	5	5	12	13
obs ₄	10	9	10	9	10	8	10	8	9	10	7
obs ₅	8	8	8	8	8	9	9	9	8	7	9

We can evaluate rank-ordering of each tree after converting leaves to WOE to understand which feature interactions have a stand-alone ranking importance (tree_3 has the highest-ranking value)

feature	tree_3	tree_1	tree_8	tree_4	tree_2	tree_7	tree_0	tree_6	tree_5	tree_9
gini	0.843	0.841	0.837	0.835	0.831	0.831	0.831	0.825	0.821	0.820

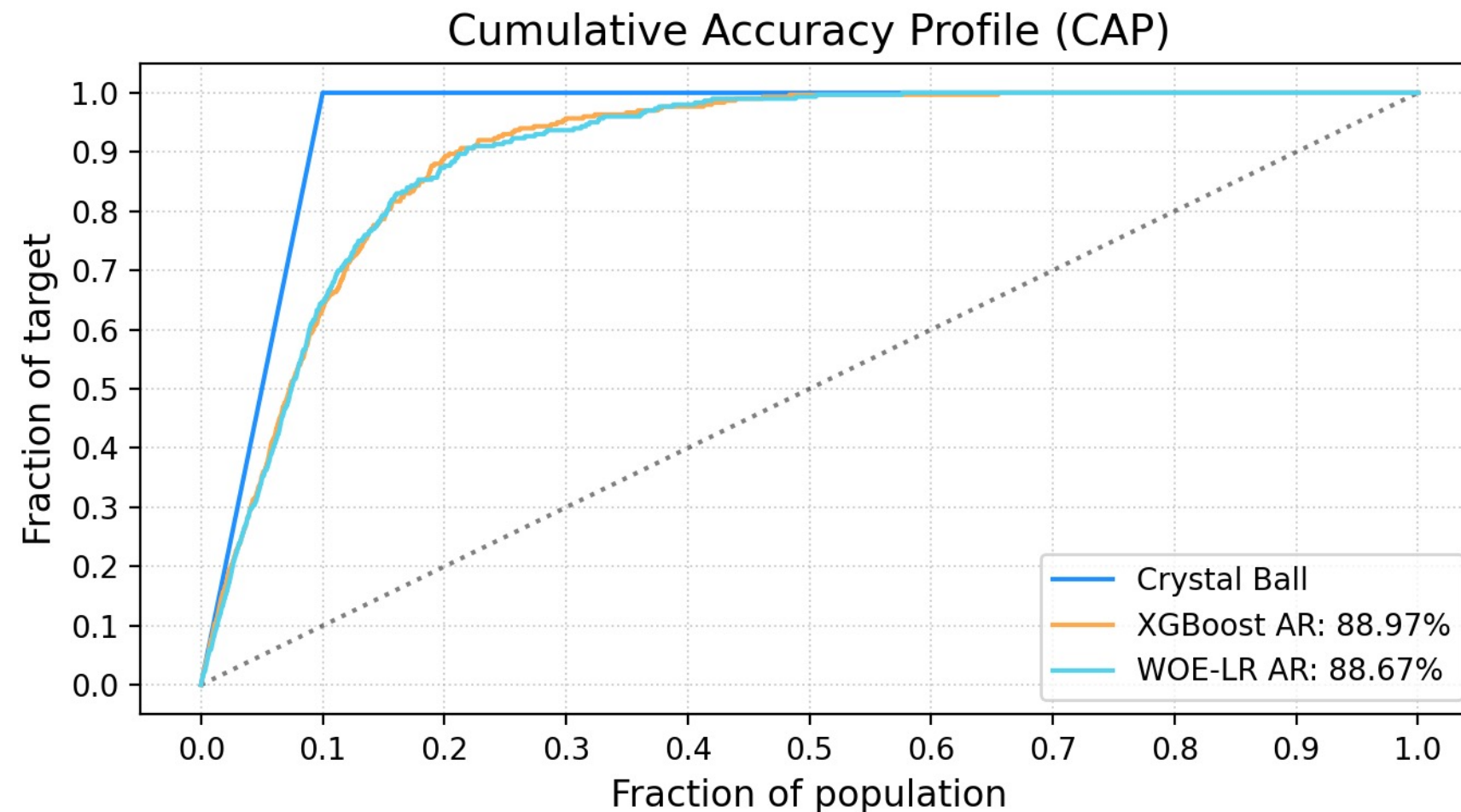
This view allows us to examine the interactions that can be included in a linear scorecard model alongside main effects

For tree_3 (the highest Gini), we can see that a useful split in index 5 allows to filter out *half of the portfolio with just 0.17% default rate*

Tree	Node	DetailedSplit	CountPct	EventRate
3	5	account_never_delinq_percent >= 98.4000015 or missing, revolving_utilization_of_unsecured_lines < 0.697415113	0.490143	0.001749
3	7	account_never_delinq_percent < 98.4000015, revolving_utilization_of_unsecured_lines < 0.503930151, external_risk_estimate < 71	0.087857	0.232520
3	8	account_never_delinq_percent < 98.4000015, revolving_utilization_of_unsecured_lines < 0.503930151, external_risk_estimate >= 71 or missing	0.168143	0.047579
3	9	account_never_delinq_percent < 98.4000015, revolving_utilization_of_unsecured_lines >= 0.503930151 or missing, external_risk_estimate < 84	0.124429	0.528129
3	10	account_never_delinq_percent < 98.4000015, revolving_utilization_of_unsecured_lines >= 0.503930151 or missing, external_risk_estimate >= 84 or missing	0.024857	0.103448
3	11	account_never_delinq_percent >= 98.4000015 or missing, revolving_utilization_of_unsecured_lines >= 0.697415113 or missing, external_risk_estimate < 61	0.011571	0.123457
3	12	account_never_delinq_percent >= 98.4000015 or missing, revolving_utilization_of_unsecured_lines >= 0.697415113 or missing, external_risk_estimate >= 61 or missing	0.093000	0.010753

Boosted Tree Interactions to WOE

A New Approach to Feature Engineering in Credit Scoring



XGBoost (100 trees)	
Metric	Value
Train Gini	92.30%
Test Gini	88.97%
Overfit	3.33%

WOE-LR (+interactions)	
Metric	Value
Train Gini	89.20%
Test Gini	88.67%
Overfit	0.53%

Note that on the test set, we can achieve nearly the same ranking performance with a linear model (main effects + interactions from GBDT) as with XGBoost

For this feature engineering pipeline, we used two Python libraries:

- [xbooster](#) to produce GBDT leaf indices
- [fastwoe](#) to convert indices to WOE values

You can find the notebook with these experiments here: [gbdt-leaf-woe.ipynb](#)